



colegio
CALASANZ
SALAMANCA

Paseo de Canalejas, 139-159
37001 Salamanca
Tel.: 923 26 79 61 | Fax: 923 27 04 54
www.calasanzsalamanca.es
CENTRO PRIVADO CONCERTADO

BOOKTRACK

CFGS DAW
CURSO 2025/2026
ALVARO BRAVO MARCOS, JUAN RODRIGUEZ LACOSTE
HUGO GAGO VICENTE
COLEGIO CALASANZ SALAMANCA

Índice

1.	Introducción	3
2.	Descripción de la aplicación	4
3.	Plan de empresa	5
4.	Tecnologías escogidas y justificación	8
5.	Diseño de la aplicación	9
5.1.	Diagramas y definición de casos de uso	9
5.2.	Diagramas de clase	12
5.3.	Modelo entidad relación y modelo de base de datos	13
6.	Arquitectura de la aplicación	15
6.1.	Estructura del proyecto	15
6.2.	Librerías externas utilizadas	16
7.	Manual de despliegue	17

1. Introducción

BookTrack es una aplicación web destinada al seguimiento personal de lecturas. El objetivo del proyecto es ofrecer una herramienta sencilla y completa para que cada usuario pueda registrar los libros que desea leer, los que tiene en curso y los que ya ha finalizado, incorporando información bibliográfica, valoración personal, comentarios y estadísticas de progreso.

El proyecto se enmarca en el módulo de Proyecto de Desarrollo de Aplicaciones Web del CFGS DAW. La memoria se estructura de acuerdo con la guía de desarrollo del centro: introducción, descripción, plan de empresa, tecnologías, diseño, arquitectura y manual de despliegue. Además, se incorpora documentación complementaria para cubrir los criterios de evaluación relacionados con análisis del sector, planificación, riesgos, presupuesto, calidad y evaluación.

La aplicación responde a una necesidad habitual de los lectores: centralizar la biblioteca personal y disponer de indicadores que permitan visualizar hábitos de lectura. Aunque existen plataformas sociales de lectura, BookTrack se plantea como una solución más directa y privada, centrada en el control personal de la información, el aislamiento de datos por usuario y la generación de estadísticas útiles para la toma de decisiones personales.

2. Descripción de la aplicación

BookTrack estará formada por una interfaz web desarrollada con Vue 3 y un backend REST desarrollado con Java 17 y Spring Boot. El usuario podrá acceder a la aplicación mediante registro e inicio de sesión. Una vez autenticado, dispondrá de un espacio personal donde podrá gestionar su biblioteca, consultar un dashboard de actividad, revisar estadísticas y acceder a un catálogo global de libros.

2.1. Funcionalidades principales

- Registro e inicio de sesión mediante JWT, con sesión stateless de 24 horas y contraseñas cifradas mediante BCrypt.

- Gestión CRUD de libros, incluyendo título, autor, género, editorial, número de páginas, nota de 1 a 10 y comentarios.
- Control de estados de lectura: pendiente, en curso y leído, con actualización automática de fechas de inicio y fin.
- Filtros por texto libre, estado, autor, género y editorial para localizar libros dentro de la biblioteca personal.
- Dashboard con libros leídos en el mes, libros leídos en el año, libros en curso, pendientes y progreso del objetivo anual.
- Estadísticas con gráfico mensual de libros leídos y distribución por géneros.
- Catálogo global donde se muestran libros registrados por todos los usuarios, sin permitir la modificación de datos ajenos.
- Objetivo anual configurable con barra de progreso y porcentaje de cumplimiento.
- Aislamiento total de datos por usuario en las operaciones privadas de biblioteca.

2.2. Flujo de navegación

El flujo de navegación parte de una pantalla pública de acceso. El visitante puede registrarse o iniciar sesión. Tras autenticarse, accede al resumen personal. Desde esta pantalla puede moverse a la biblioteca, al detalle de cada libro, a las estadísticas, al catálogo global y a la configuración del objetivo anual. El diseño pretende que las acciones principales estén disponibles con pocos clics y que el usuario entienda de forma inmediata el estado de sus lecturas.

```

Visitante
|-- Registro / Login
    |-- Dashboard personal
        |-- Biblioteca
            |-- Alta de libro
            |-- Edición / eliminación
            |-- Cambio de estado
        |-- Estadísticas
        |-- Catálogo global
        |-- Objetivo anual

```

2.3. Requisitos funcionales

Código	Requisito
RF01	El usuario podrá registrarse con nombre, usuario, correo y contraseña.
RF02	El usuario podrá iniciar sesión y recibir un token JWT válido.
RF03	El usuario podrá crear, consultar, editar y eliminar libros de su biblioteca.
RF04	El usuario podrá cambiar el estado de un libro entre pendiente, en curso y leído.
RF05	El sistema asignará fechas automáticas cuando un libro pase a en curso o leído.
RF06	El usuario podrá filtrar libros por texto, estado, autor, género y editorial.
RF07	El dashboard mostrará indicadores mensuales, anuales y de objetivo lector.
RF08	El sistema generará estadísticas mensuales y por género.
RF09	El catálogo global mostrará libros registrados por todos los usuarios.
RF10	El usuario podrá modificar su objetivo anual de lectura.

2.4. Requisitos no funcionales

Código	Requisito
RNF01	La aplicación se desplegará mediante Docker y Docker Compose.
RNF02	El backend utilizará arquitectura por capas Controller - Service - Repository.
RNF03	Las contraseñas se almacenarán cifradas con BCrypt.
RNF04	La autenticación será stateless mediante JWT.
RNF05	La aplicación deberá funcionar en navegadores modernos y en resoluciones habituales de escritorio.
RNF06	La base de datos usará MySQL 8.0 y persistencia mediante volumen Docker.
RNF07	El código deberá estar organizado para facilitar mantenimiento y ampliaciones.

3. Plan de empresa

3.1. Análisis del sector

El sector en el que se sitúa BookTrack es el desarrollo de software web, especialmente aplicaciones SaaS orientadas a la productividad personal, gestión de contenidos y análisis de hábitos. En este ámbito se encuentran empresas de consultoría tecnológica, startups de producto digital, proveedores de plataformas SaaS y empresas especializadas en aplicaciones educativas o culturales.

Tipo de empresa	Características organizativas	Producto o servicio
Consultoría tecnológica	Equipos por proyectos, analistas, desarrolladores y soporte.	Desarrollo a medida de aplicaciones web y APIs.
Startup SaaS	Estructura reducida, producto propio, evolución iterativa.	Plataformas de suscripción y servicios cloud.
Empresa de software educativo/cultural	Departamentos de contenido, desarrollo y atención a usuarios.	Herramientas de aprendizaje, lectura y gestión cultural.
Proveedor cloud o DevOps	Equipos de infraestructura, seguridad y operaciones.	Alojamiento, contenedores, CI/CD y monitorización.

3.2. Empresa tipo y departamentos

La empresa tipo para un producto como BookTrack sería una pequeña empresa tecnológica de producto propio. Su estructura combinaría perfiles técnicos, de negocio y de soporte. La organización por departamentos permite separar responsabilidades y asegurar que el producto evolucione de forma controlada.

Departamento	Funciones
Dirección y gestión	Definición de estrategia, planificación, control económico y coordinación del equipo.
Desarrollo frontend	Diseño de interfaz, componentes Vue, conexión con API y experiencia de usuario.
Desarrollo backend	API REST, seguridad, lógica de negocio, persistencia y pruebas de servicios.
Base de datos y DevOps	Modelado de datos, Docker, despliegue, copias de seguridad y disponibilidad.
Marketing y atención al cliente	Captación de usuarios, documentación, soporte y análisis de satisfacción.
Administración	Facturación, obligaciones fiscales, contratos, protección de datos y gestión laboral.

3.3. Necesidades demandadas

Las necesidades más demandadas en las empresas del sector son la digitalización de procesos, el desarrollo de aplicaciones seguras, la integración de bases de datos, la autenticación de usuarios, el despliegue automatizado y la disponibilidad de métricas de uso. Los clientes buscan soluciones que funcionen de forma estable, sean fáciles de mantener y puedan evolucionar sin rehacer toda la arquitectura.

En el caso concreto de BookTrack, la demanda se concreta en una aplicación que permita registrar información personal de lectura, consultar datos con rapidez, obtener estadísticas y mantener la privacidad de cada usuario. La solución debe combinar facilidad de uso con una base técnica sólida.

3.4. Oportunidades de negocio

Las oportunidades previsibles se relacionan con el crecimiento de las aplicaciones de productividad personal, la lectura digital, los clubes de lectura y la analítica de hábitos. BookTrack podría evolucionar hacia un modelo freemium, incorporar recomendaciones, exportación de datos, integración con APIs bibliográficas o funcionalidades colaborativas para grupos de lectura y centros educativos.

3.5. Tipo de proyecto requerido y características específicas

El proyecto requerido es una aplicación web full stack con backend REST, frontend SPA, base de datos relacional y despliegue en contenedores. Debe incluir autenticación segura, operaciones CRUD, filtros, panel de control, estadísticas y separación de datos por usuario. La solución se ajusta a un producto SaaS ligero, preparado para funcionar en local o en un entorno de hosting de contenedores.

3.6. Obligaciones fiscales, laborales y de prevención

En un escenario real de explotación empresarial, el proyecto debería cumplir obligaciones fiscales como alta censal mediante modelos 036 o 037, facturación conforme a la normativa vigente, declaración de IVA e IRPF o impuesto de sociedades según la forma jurídica. También sería necesario cumplir la normativa de protección de datos, ya que la aplicación trata datos identificativos y hábitos de lectura de usuarios.

Desde el punto de vista laboral, si existieran trabajadores contratados, deberían gestionarse contratos, alta en Seguridad Social, cotizaciones y cumplimiento del Estatuto de los Trabajadores. En prevención de riesgos laborales, al tratarse de actividad informática, los riesgos principales serían ergonómicos, fatiga visual, sedentarismo, carga mental y riesgos eléctricos de baja intensidad. Deben aplicarse pausas, correcta iluminación, ergonomía del puesto y formación básica en PRL.

3.7. Ayudas o subvenciones

Para un proyecto de digitalización o software pueden considerarse programas como Kit Digital, ayudas de Red.es, convocatorias del CDTI para innovación, programas ENISA para emprendimiento innovador, fondos europeos vinculados a digitalización y ayudas autonómicas de apoyo a empresas tecnológicas. Estas líneas podrían financiar infraestructura, desarrollo, consultoría, ciberseguridad o acciones de crecimiento.

3.8. Guion de trabajo

Fase	Contenido
Análisis	Definición de requisitos, alcance, usuarios, casos de uso y riesgos iniciales.
Diseño	Modelo de datos, arquitectura, estructura del proyecto y prototipos de interfaz.
Implementación backend	Entidades JPA, repositorios, servicios, controladores y seguridad JWT.
Implementación frontend	Vistas Vue, componentes, servicios Axios, filtros y dashboard.
Despliegue	Dockerfiles, docker-compose, variables de entorno y pruebas de arranque.
Evaluación	Pruebas, revisión de requisitos, incidencias, documentación y preparación de defensa.

3.9. Viabilidad técnica, económica y financiera

La viabilidad técnica es alta porque las tecnologías elegidas son estables, documentadas y ampliamente utilizadas en entornos reales. Java 17 y Spring Boot ofrecen robustez en backend; Vue 3 y Vite permiten un frontend rápido; MySQL es una base de datos consolidada; Docker facilita el despliegue reproducible.

Concepto	Detalle	Coste estimado
Equipos informáticos	Tres equipos de desarrollo ya disponibles. Coste imputado de amortización.	1.500 €
Licencias de desarrollo	Docker, MySQL Community, Java, Maven, Vue y Vite. Uso gratuito.	0 €
Dominio	Dominio anual para publicación del producto.	15 €
Hosting de contenedores/VPS	Servidor básico para ejecutar frontend, backend y MySQL.	120 € / año
Diseño y desarrollo	Horas de análisis, diseño, implementación y pruebas del equipo.	2.000 €
Mantenimiento inicial	Correcciones, copias de seguridad y soporte básico.	300 €
Total estimado		3.935 €

Las necesidades de financiación son moderadas porque la mayoría de herramientas son gratuitas o de código abierto. La financiación se destinaría principalmente a infraestructura, dominio, soporte y evolución del producto. Podría cubrirse mediante aportación propia, ayudas a digitalización o financiación para emprendimiento tecnológico.

3.10. Documentación referida al proyecto y evaluación

3.10.1. Planificación y ejecución

Semana	Actividad	Recursos
1-2	Análisis de requisitos, alcance y prototipos.	Equipo completo, guía del proyecto, editor de diagramas.
Semana	Actividad	Recursos
3-5	Desarrollo del backend: entidades, repositorios, servicios, seguridad y API.	Java 17, Spring Boot, Maven, MySQL.
6-8	Implementación del frontend y conexión con API.	Vue 3, Vite, Axios, navegador.
9-10	Dashboard, estadísticas, filtros y catálogo global.	Backend, frontend, datos de prueba.
11	Pruebas, correcciones, despliegue Docker y revisión de calidad.	Docker, GitHub, checklist de requisitos.
12	Memoria final, manual de despliegue y preparación de defensa.	PDF, repositorio, capturas y demostración.

3.10.2. Permisos y autorizaciones

Para ejecutar el proyecto en entorno académico no se requieren autorizaciones administrativas específicas, pero sí debe respetarse la propiedad intelectual del código, mantener el repositorio bajo control del equipo y evitar el uso de datos personales reales innecesarios. En explotación pública se necesitaría aviso legal, política de privacidad, política de cookies si procede, términos de uso y consentimiento para el tratamiento de datos personales.

3.10.3. Riesgos y plan de prevención

Riesgo	Probabilidad	Impacto	Medida preventiva
Pérdida de datos de MySQL	Media	Alta	Uso de volumen persistente y copias periódicas.
Fallo de autenticación JWT	Baja	Alta	Pruebas de login, expiración y acceso a endpoints protegidos.
Acceso a libros de otro usuario	Baja	Alta	Validar propiedad en BookService antes de devolver o modificar datos.
Errores en despliegue Docker	Media	Media	Documentar comandos, variables y healthcheck.
Incompatibilidad frontend-backend	Media	Media	Definir endpoints, probar Axios y revisar CORS.
Retraso en implementación	Media	Media	Planificar tareas por semanas y priorizar requisitos obligatorios.
Vulnerabilidades por contraseñas	Baja	Alta	BCrypt, no almacenar contraseñas en claro y revisar logs.
Fatiga visual o postural	Media	Media	Pausas, ergonomía, iluminación y organización de tiempos.

3.10.4. Procedimientos de actuación

El procedimiento de ejecución se apoya en Git y ramas de trabajo. Cada funcionalidad se implementa en una rama, se prueba en local, se revisa y se integra en la rama principal. Las incidencias se registran con descripción, fecha, responsable, prioridad y estado. Los cambios que afecten al modelo de datos, seguridad o endpoints deben documentarse en la memoria o en el README del repositorio.

3.10.5. Indicadores de calidad

Indicador	Criterio de aceptación
Autenticación	Registro y login funcionan; el token permite acceder a endpoints protegidos y caduca correctamente.
Aislamiento de datos	Un usuario no puede consultar, editar ni borrar libros de otro usuario.
CRUD de libros	Crear, listar, editar y eliminar funciona sin errores visibles.
Estados de lectura	Las transiciones actualizan estado y fechas de forma coherente.
Filtros	La biblioteca responde a búsquedas por texto, estado, autor, género y editorial.
Dashboard	Los totales mensuales, anuales y de objetivo coinciden con los datos almacenados.
Despliegue	docker-compose up -d --build levanta los tres servicios sin errores.
Usabilidad	La interfaz permite completar las acciones principales sin instrucciones externas.

3.10.6. Matriz de trazabilidad requisitos-implementación

Req.	Implementación	Prueba asociada
RF01	AuthController + AuthService + UserRepository	Registrar usuario nuevo y comprobar persistencia.
RF02	JwtTokenProvider + SecurityConfig + JwtAuthenticationFilter	Iniciar sesión y acceder a endpoint protegido con token.
RF03	BookController + BookService + BookRepository	Crear, listar, editar y borrar un libro.
RF04	PUT /api/books/{id}/status	Cambiar libro de PENDING a IN_PROGRESS y READ.
RF05	Lógica de BookService sobre startDate y endDate	Verificar fechas automáticas tras cambios de estado.
RF06	Consultas filtradas en BookRepository	Buscar por texto, autor, género, editorial y estado.
RF07	DashboardController + DashboardService	Comparar resumen con datos de libros.
RF08	/api/dashboard/statistics	Comprobar gráfico mensual y distribución por géneros.
RF09	CatalogController + CatalogService	Visualizar catálogo global sin modificar libros ajenos.
RF10	PUT /api/dashboard/goal	Actualizar objetivo y verificar barra de progreso.

3.10.7. Evaluación de incidencias y gestión de cambios

Las incidencias se evaluarán según prioridad alta, media o baja. Una incidencia alta bloquea el uso básico de la aplicación, como login, CRUD o despliegue. Una incidencia media afecta a funcionalidades relevantes, pero no impide el funcionamiento general. Una incidencia baja corresponde a mejoras visuales, textos o ajustes menores. Cada incidencia se registra, se asigna a un responsable, se corrige en una rama y se verifica antes de cerrar.

La gestión de cambios se realizará mediante control de versiones en Git. Los cambios de alcance deberán justificarse, indicando motivo, impacto en recursos, impacto en planificación y pruebas necesarias. No se aceptarán cambios que contradigan requisitos aprobados salvo autorización del tutor.

3.10.8. Participación de usuarios y cumplimiento del pliego

La participación de usuarios se realizará mediante pruebas con usuarios del entorno académico. Se recogerán observaciones sobre claridad de interfaz, facilidad para añadir libros, utilidad del dashboard y comprensión de estadísticas. Esta información se usará para corregir errores y priorizar mejoras.

El cumplimiento del pliego se garantizará mediante una lista de verificación basada en requisitos funcionales, requisitos no funcionales, despliegue, documentación y defensa. Antes de la entrega se comprobará que el repositorio contiene backend, frontend, base de datos, Dockerfiles, docker-compose, memoria y etiqueta final de depósito.

4. Tecnologías escogidas y justificación

Capa	Tecnología	Justificación
Backend	Java 17 + Spring Boot 3.4.0	Permite construir APIs REST robustas, mantenibles y con integración sencilla con seguridad y persistencia.
Seguridad	Spring Security 6 + JWT (JJWT 0.12.6)	Modelo stateless adecuado para una SPA. El token evita sesiones en servidor y facilita despliegues escalables.
Frontend	Vue 3 + Vite 5 + Axios	Vue facilita el desarrollo por componentes; Vite acelera el entorno de desarrollo; Axios simplifica llamadas HTTP.
Persistencia	MySQL 8.0 + Spring Data JPA/Hibernate	Base de datos relacional estable. JPA reduce código repetitivo y permite mapear entidades Java a tablas.
Build	Maven 3.9.6	Gestiona dependencias, ciclo de vida del backend y empaquetado reproducible.

Contenedores	Docker + Docker Compose	Permite levantar base de datos, backend y frontend con una configuración común para todo el equipo.
--------------	-------------------------	---

La elección de estas tecnologías responde a criterios de aprendizaje, fiabilidad y posibilidad de despliegue real. La combinación Spring Boot - Vue - MySQL permite separar claramente frontend, backend y datos, mientras que Docker reduce problemas de configuración entre equipos.

5. Diseño de la aplicación

5.1. Diagramas y definición de casos de uso

Los casos de uso describen las acciones que pueden realizar los actores principales. En esta versión se mantienen las imágenes antiguas de los diagramas para conservar el diseño original del anteproyecto: un diagrama orientado al administrador y otro orientado al usuario general.

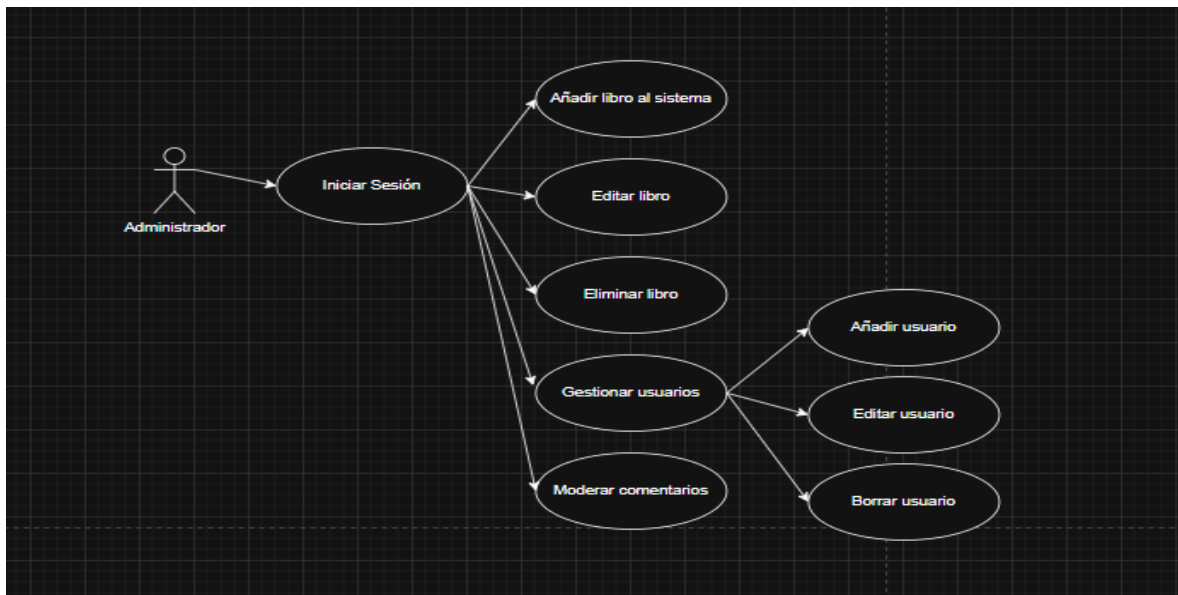


Figura 1. Diagrama antiguo de casos de uso del administrador.

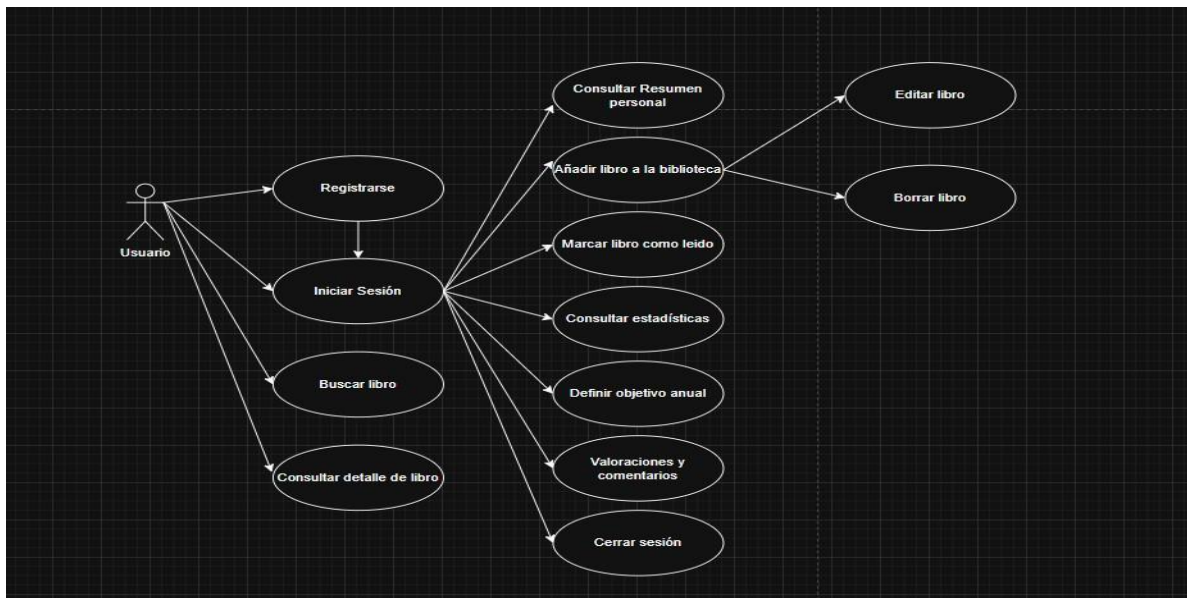


Figura 2. Diagrama antiguo de casos de uso del usuario.

CU	Nombre	Actor	Descripción resumida
CU01	Registro	Visitante	Permite crear una cuenta de usuario con datos únicos y contraseña cifrada.
CU02	Inicio de sesión	Usuario	Permite autenticarse y recibir un token JWT para acceder a recursos protegidos.
CU03	Gestionar libro	Usuario	Permite crear, listar, editar y eliminar libros propios.
CU04	Cambiar estado	Usuario	Permite pasar un libro por los estados pendiente, en curso y leído.
CU05	Consultar dashboard	Usuario	Muestra resumen mensual, anual, pendientes, en curso y objetivo.
CU06	Consultar estadísticas	Usuario	Muestra libros leídos por mes y distribución por géneros.
CU07	Actualizar objetivo anual	Usuario	Permite establecer la meta anual de libros leídos.
CU08	Consultar catálogo global	Usuario	Permite ver libros registrados por otros usuarios en modo consulta.

Definición de casos de uso principales

CU01 - Registro de usuario

Actor: Visitante. Precondición: El visitante no dispone de cuenta registrada.

Postcondición: El usuario queda almacenado en la tabla users con contraseña cifrada.

1. Accede al formulario de registro.
2. Introduce nombre, usuario, correo y contraseña.
3. El sistema valida que usuario y correo sean únicos.
4. El sistema cifra la contraseña y guarda la cuenta.

CU02 - Inicio de sesión

Actor: Usuario. Precondición: El usuario está registrado. Postcondición: El sistema devuelve un JWT válido durante 24 horas.

1. Introduce usuario y contraseña.
2. El backend valida credenciales con Spring Security.
3. JwtTokenProvider genera el token.
4. El frontend almacena el token para peticiones posteriores.

CU03 - Gestión de libros

Actor: Usuario. Precondición: El usuario está autenticado. Postcondición: La biblioteca del usuario queda actualizada.

1. Accede a biblioteca.
2. Crea, edita o elimina un libro.
3. BookService valida la propiedad del recurso.
4. BookRepository persiste los cambios en MySQL.

CU04 - Cambio de estado

Actor: Usuario. Precondición: El libro pertenece al usuario. Postcondición: El estado y las fechas se actualizan de forma coherente.

1. Selecciona un libro.

2. Elige nuevo estado.
3. El servicio aplica reglas de transición.
4. Se guarda startDate o endDate cuando corresponda.

CU05 - Dashboard y estadísticas

Actor: Usuario. Precondición: Existen libros asociados al usuario.

Postcondición: Se muestran métricas agregadas en pantalla.

1. El frontend solicita /api/dashboard/summary o /statistics.
2. DashboardService consulta los libros del usuario.
3. El backend calcula totales y agrupaciones.
4. La interfaz muestra tarjetas, gráficos y barra de progreso.

5.2. Diagramas de clases

El siguiente diagrama resume las clases principales del backend y su relación con entidades, servicios, repositorios y seguridad. Se sigue una arquitectura por capas: los controladores reciben peticiones, los servicios aplican reglas de negocio, los repositorios acceden a MySQL y las clases de seguridad protegen los endpoints.

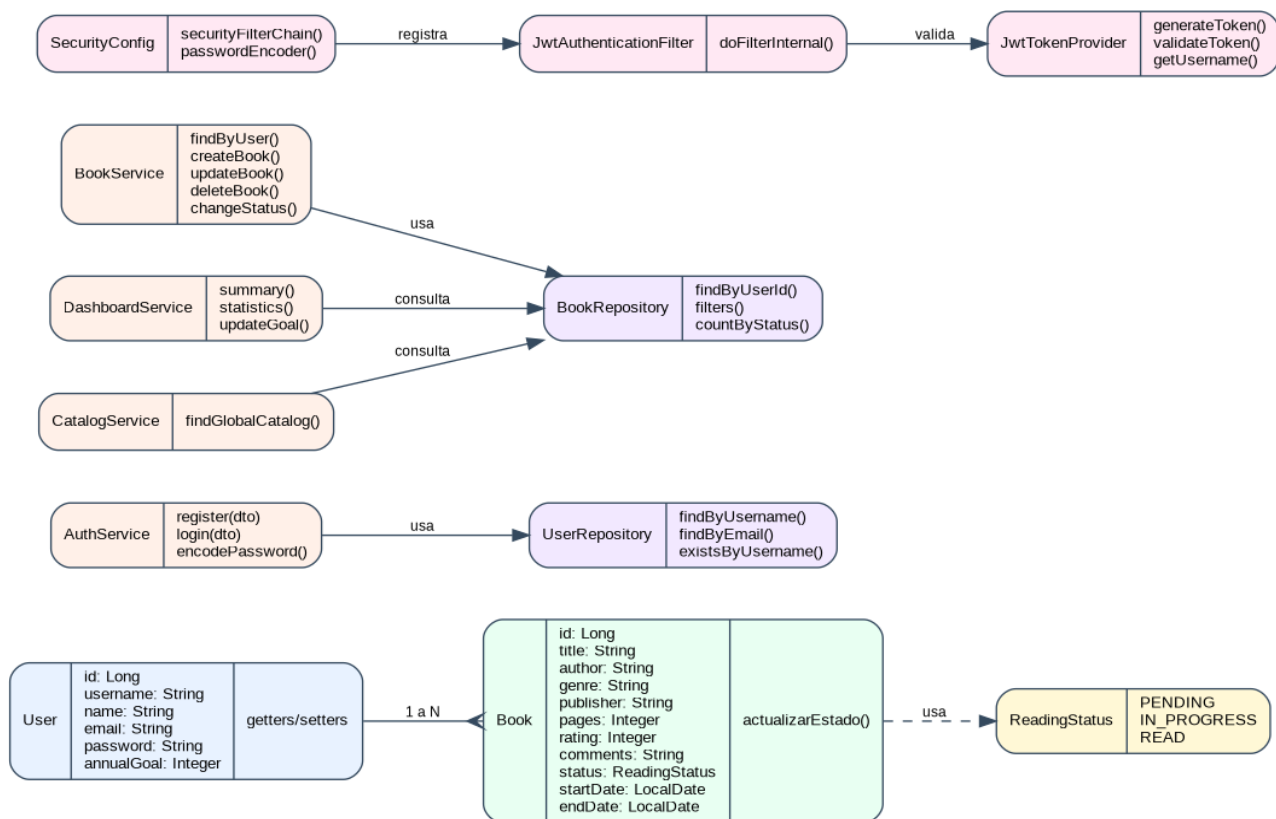


Figura 3. Diagrama de clases principales de BookTrack.

Descripción de qué hace cada clase del diagrama

Clase	Qué hace
User	Representa a cada usuario registrado. Guarda credenciales, datos básicos y objetivo anual. Es la entidad propietaria de los libros.
Book	Representa un libro de la biblioteca. Contiene metadatos, valoración, comentarios, estado y fechas de lectura.
ReadingStatus	Enumera los estados posibles de un libro: PENDING, IN_PROGRESS y READ. Facilita validar el flujo de lectura.
AuthService	Gestiona registro e inicio de sesión. Comprueba duplicados, cifra contraseñas y coordina la generación de JWT.
BookService	Contiene la lógica de negocio de libros: altas, edición, borrado, filtros, validación de propiedad y cambios de estado.
DashboardService	Calcula indicadores del dashboard: libros del mes, del año, pendientes, en curso y progreso del objetivo anual.
CatalogService	Obtiene el catálogo global de libros de todos los usuarios en modo lectura.
UserRepository	Accede a la tabla users mediante Spring Data JPA y aporta búsquedas por username y email.
BookRepository	Accede a la tabla books, ejecutando consultas por usuario, estado, autor, género, editorial y fechas.
JwtTokenProvider	Genera, valida y analiza tokens JWT. Extrae el username y controla la validez temporal del token.
JwtAuthenticationFilter	Intercepta cada petición HTTP, lee el token Bearer y crea la autenticación en el contexto de seguridad.
SecurityConfig	Define la configuración de seguridad: endpoints públicos, endpoints protegidos, filtro JWT, CORS y PasswordEncoder.

Diagrama de capas

```
Petición HTTP
|
v
Controller  -> valida entrada y expone endpoints REST
|
v
Service      -> aplica reglas de negocio y seguridad de propiedad
|
v
Repository   -> ejecuta operaciones JPA contra MySQL
|
v
MySQL        -> almacena usuarios y libros
```

La separación por capas evita mezclar responsabilidades. Por ejemplo, un controlador no debería construir consultas complejas ni aplicar reglas de negocio; esas tareas se delegan en servicios y repositorios. Esta organización facilita pruebas, mantenimiento y ampliación futura.

5.3. Modelo entidad-relación y modelo de base de datos

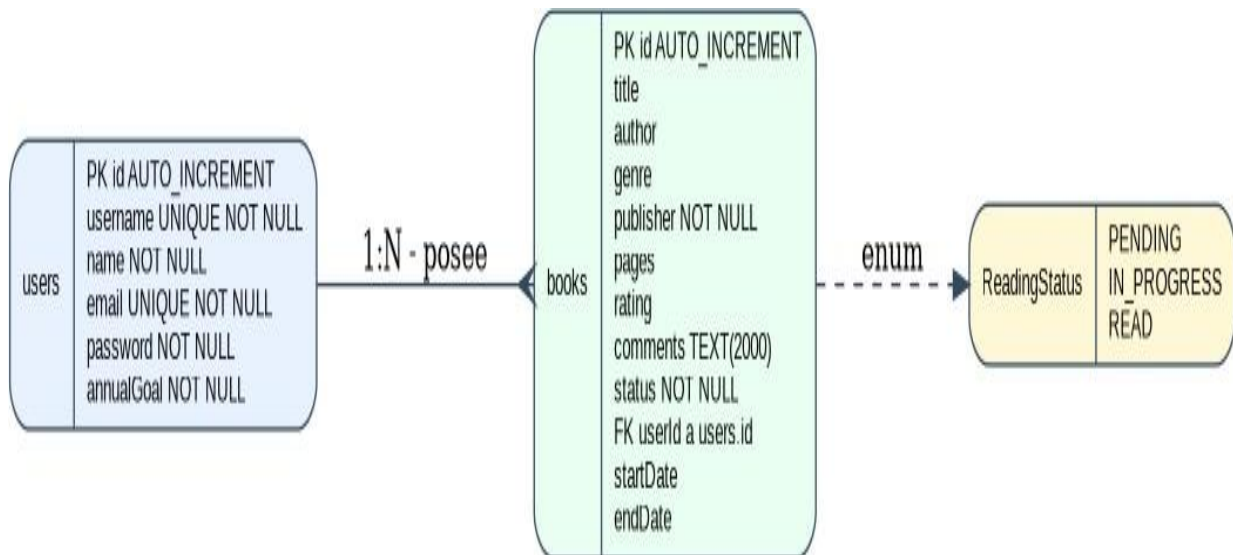


Figura 4. Modelo entidad-relación y tablas principales.

El modelo de datos se compone de dos tablas principales: users y books. La relación es uno a muchos, porque un usuario puede tener muchos libros, pero cada libro pertenece a un único usuario. La clave foránea userId en books permite garantizar el aislamiento de datos por usuario.

Tabla	Campos principales	Descripción
users	id, username, name, email, password, annualGoal	Almacena la información de autenticación y configuración básica de cada usuario. username y email son únicos.
books	id, title, author, genre, publisher, pages, rating, comments, status, userId, startDate, endDate	Almacena los libros y su estado de lectura. userId referencia a users.id.

6. Arquitectura de la aplicación

BookTrack utiliza una arquitectura cliente-servidor separada en tres bloques: frontend Vue, backend Spring Boot y base de datos MySQL. La comunicación entre frontend y backend se realiza mediante una API REST sobre HTTP. La comunicación entre backend y base de datos se realiza mediante JDBC, Spring Data JPA e Hibernate.



La infraestructura con Docker Compose orquesta tres servicios. El servicio db usa la imagen mysql:8.0, define un volumen persistente db_data y dispone de healthcheck con mysqladmin ping. El backend se construye con un Dockerfile multi-stage: una fase de build con Maven y una fase final con eclipse-temurin:17-jre-alpine, que reduce el tamaño de la imagen. El frontend se construye con su propio Dockerfile y depende del backend.

Los contenedores se comunican mediante la red interna creada automáticamente por Docker Compose. El backend no necesita conocer una IP fija de MySQL, sino el nombre del servicio db. La dependencia del backend respecto a db incluye la condición service_healthy, de manera que Spring Boot arranca cuando MySQL ya responde correctamente. El volumen db_data conserva la información aunque el contenedor de base de datos se elimine y se vuelva a crear.

6.1. Estructura del proyecto

```

BookTrack/
├── backend/
│   ├── src/main/java/.../controller
│   ├── src/main/java/.../service
│   ├── src/main/java/.../repository
│   ├── src/main/java/.../model
│   ├── src/main/java/.../security
│   ├── src/main/resources/application.properties
│   ├── pom.xml
│   └── Dockerfile
├── frontend/
│   ├── src/components
│   ├── src/views
│   ├── src/services
│   ├── src/router
│   ├── package.json
│   └── Dockerfile
└── docker-compose.yml

```

La estructura del backend separa controladores, servicios, repositorios, modelos y seguridad. En el frontend se separan vistas, componentes reutilizables, servicios de comunicación y rutas. En la raíz del repositorio se ubican los archivos de despliegue para facilitar que cualquier miembro del equipo pueda levantar el entorno completo.

6.2. Librerías externas utilizadas

Entorno	Librería / dependencia	Uso
Backend	spring-boot-starter-web	Creación de API REST y controladores HTTP.
Backend	spring-boot-starter-security	Autenticación, autorización y protección de endpoints.
Backend	jjwt 0.12.6	Generación y validación de tokens JWT.
Backend	spring-boot-starter-data-jpa	Persistencia con repositorios JPA.
Backend	mysql-connector-j	Conexión JDBC con MySQL 8.0.
Backend	lombok	Reducción de código repetitivo en entidades y DTO.
Frontend	vue	Construcción de interfaz por componentes.
Frontend	vite	Servidor de desarrollo y empaquetado rápido.
Frontend	axios	Peticiones HTTP al backend.

7. Manual de despliegue

7.1. Requisitos previos

- Docker y Docker Compose instalados.
- Puerto 3000 disponible para el frontend.

- Puerto 8081 disponible para el backend.
- Puerto 3306 libre si se desea exponer MySQL localmente.
- Repositorio del proyecto descargado en el equipo.

7.2. Variables de entorno

Variable	Servicio	Descripción
SPRING_DATASOURCE_URL	backend	URL JDBC hacia el servicio db. Ejemplo: jdbc:mysql://db:3306/booktrack
SPRING_DATASOURCE_USERNAME	backend	Usuario de conexión a MySQL.
SPRING_DATASOURCE_PASSWORD	backend	Contraseña de conexión a MySQL.
SPRING_JPA_HIBERNATE_DDL_AUTO	backend	Valor update para que Hibernate actualice el esquema automáticamente.
MYSQL_DATABASE	db	Nombre de la base de datos inicial.
MYSQL_ROOT_PASSWORD	db	Contraseña del usuario root de MySQL.

7.3. Despliegue con Docker Compose

Desde la raíz del proyecto se ejecuta el siguiente comando para construir y arrancar todos los servicios:

```
docker-compose up -d --build
```

El resultado esperado es que se creen tres contenedores: db, backend y frontend. El contenedor db inicializa MySQL y activa el healthcheck. Cuando la base de datos está saludable, se inicia backend. Finalmente arranca frontend, que queda accesible desde el navegador.

```
# Comprobar estado de contenedores
docker compose ps

# Ver logs del backend
docker compose logs -f backend

# Parar servicios
docker compose down

# Parar y eliminar volumen de datos, solo si se desea reiniciar la BBDD
docker compose down -v
```

7.4. Imágenes Docker Hub y comandos individuales

En la entrega final debe indicarse el nombre de las imágenes publicadas en Docker Hub y la línea de comandos para ejecutar cada contenedor. Si se publican con el usuario del equipo, se propone la siguiente nomenclatura:

```
docker build -t usuario/booktrack-backend:1.0 ./backend
docker build -t usuario/booktrack-frontend:1.0 ./frontend

docker push usuario/booktrack-backend:1.0
docker push usuario/booktrack-frontend:1.0
```

Comandos de ejecución individual orientativos:

```
docker network create booktrack-net

docker run -d --name booktrack-db --network booktrack-net -e MYSQL_DATABASE=booktrack -e

docker run -d --name booktrack-backend --network booktrack-net -p 8081:8081 -e SPRING_DATASO

docker run -d --name booktrack-frontend --network booktrack-net -p 3000:3000 usuario/booktra
```

7.5. Despliegue local sin Docker

```
# Backend
cd backend
mvn clean package
java -jar target/*.jar

# Frontend
cd frontend
npm install
npm run dev
```

El despliegue local exige tener MySQL instalado y configurar `application.properties` con las credenciales correctas. Para la entrega se recomienda Docker Compose porque reduce errores y garantiza que todos los servicios se ejecuten con versiones controladas.